

From Zero to Quant: A Complete Walkthrough of the Natural Gas Storage & Credit Risk Project

A beginner-friendly guide to everything built in this project — natural gas price modeling, storage contract pricing, loan default prediction, and FICO score bucketing

Prepared for Dharmi

July 20, 2026

Contents

1	How to Read This Guide	4
2	Part 0: Foundations	5
2.1	0.1 What Is a Commodity, and Why Does It Get “Traded”?	5
2.2	0.2 What Is a Storage Contract?	5
2.3	0.3 What Is a CSV File?	5
2.4	0.4 A Very Short Introduction to Python, pandas, and “Functions”	6
2.5	0.5 What Does It Mean to “Train a Model”?	6
2.6	0.6 Two Kinds of Prediction Problem: Regression vs. Classification	7
2.7	0.7 Interpolation vs. Extrapolation	7
2.8	0.8 A Few Basic Statistics Ideas	7
2.9	0.9 What “Fitting” a Model Actually Means, Mechanically	8
2.10	0.10 Overfitting, Underfitting, and Train/Test Splits	8
2.11	0.11 A Note on Reading Python Syntax	9
3	Part One: Estimating Natural Gas Prices	10
3.1	1.1 The Business Problem, in Plain Language	10
3.2	1.2 Looking at the Raw Data First	10
3.3	1.3 Why Does Natural Gas Have a Seasonal Pattern?	10
3.4	1.4 Building the Trend Line: Linear Regression from Scratch	10
3.5	1.5 Isolating the Seasonal Pattern	11
3.6	1.6 Putting Trend and Seasonality Back Together	11
3.7	1.7 Two Different Rules for Two Different Situations	12
3.8	1.8 Reading the Charts	12
3.9	1.9 Sanity-Checking the Model	13
3.10	1.10 Why This Approach, and What Its Limits Are	13
3.11	1.11 Frequently Asked Questions	14
4	Part Two: Pricing a Storage Contract	16
4.1	2.1 From “Estimate a Price” to “Value a Deal”	16
4.2	2.2 The Core Idea: Add Up Every Cash Flow	16

4.3	2.3 A Worked Example First (No Code)	16
4.4	2.4 Why Multiple Injection and Withdrawal Dates Make This Harder	17
4.5	2.5 Building One Combined Timeline of Events	17
4.6	2.6 Walking Through the Timeline and Tracking Storage	17
4.7	2.7 Calculating the Storage Fee	18
4.8	2.8 The Final Number	18
4.9	2.9 Testing the Function	18
4.10	2.10 Aligning with the Course's Official Reference Answer	19
4.11	2.11 A Second Worked Example, Step by Step	19
4.12	2.12 Frequently Asked Questions	20
5	Part Three: Predicting Loan Defaults and Expected Loss	21
5.1	3.1 The Business Problem	21
5.2	3.2 Two Different Numbers: PD and Expected Loss	21
5.3	3.3 The Data	21
5.4	3.4 Why This Is a "Classification" Problem	22
5.5	3.5 Model One: Logistic Regression, Explained from the Ground Up	22
5.6	3.6 Feature Engineering: Why Ratios Instead of Raw Numbers	23
5.7	3.7 Model Two: Random Forests, Explained from the Ground Up	23
5.8	3.8 Measuring How Good a Model Is: AUC	23
5.9	3.9 The Expected Loss Function	24
5.10	3.10 Testing the Function	24
5.11	3.11 An Important Honest Caveat	25
5.12	3.12 A Fully Worked Numeric Example	25
5.13	3.13 Frequently Asked Questions	25
6	Part Four: Turning FICO Scores into Ratings	27
6.1	4.1 The Business Problem	27
6.2	4.2 What Is Quantization, and Why Not Just Use Equal-Width Buckets?	27
6.3	4.3 The Log-Likelihood Approach, Explained Without the Formula First	27
6.4	4.4 The Real Challenge: Choosing <i>Where</i> to Put the Boundaries	28
6.5	4.5 Dynamic Programming, Explained from Absolute Scratch	28
6.6	4.6 Applying Dynamic Programming to the Bucket Problem	29
6.7	4.7 Backtracking: Recovering the Actual Boundaries	30
6.8	4.8 From Boundaries to a Usable Rating Function	30
6.9	4.9 Checking the Result Makes Sense	31
6.10	4.10 An Honest Caveat About Sparse Data	31
6.11	4.11 Aligning with the Course's Official Reference Answer	31
6.12	4.12 A Tiny, Fully-Worked Toy Example	32
6.13	4.13 Frequently Asked Questions	32
7	Part Five: How It All Fits Together	34
7.1	5.1 The Gas Desk's Story, Start to Finish	34
7.2	5.2 The Credit Risk Story, Start to Finish	34
7.3	5.3 The Common Thread	35
8	Glossary	36

9	Closing Notes	39
10	Appendix: Full Code Listings	40
10.1	A.1 Natural Gas Price Estimation Model	40
10.2	A.2 Storage Contract Pricing	42
10.3	A.3 Loan Default Probability & Expected Loss Model	43
10.4	A.4 FICO Score Bucketing (Dynamic Programming)	46
11	Appendix B: Python & Library Quick Reference	49
11.1	B.1 pandas	49
11.2	B.2 NumPy	49
11.3	B.3 scikit-learn	49
11.4	B.4 Core Python	50

1 How to Read This Guide

This document explains, from first principles, everything behind four pieces of code built for a quantitative-research-style project involving a natural gas trading desk and a retail bank’s loan book. It assumes **no prior background** in trading, statistics, or programming beyond curiosity. Every technical term is defined the first time it’s used, and every piece of code is walked through line by line.

The guide is organized into six parts:

1. **Foundations** — the basic ideas (what a commodity is, what a CSV file is, what a function is, what “training a model” even means) that everything else is built on.
2. **Part One: Estimating Natural Gas Prices** — building a model that can guess the price of natural gas on any date, past or future.
3. **Part Two: Pricing a Storage Contract** — turning those price estimates into an actual dollar value for a real trading deal.
4. **Part Three: Predicting Loan Defaults** — building a model that estimates how likely a borrower is to fail to repay a loan, and what that costs the bank.
5. **Part Four: Turning FICO Scores into Ratings** — a technique called “quantization” that turns a continuous number (a credit score) into clean categories a bigger model can use.
6. **Glossary and Full Code Appendix** — every term defined in one place, and the complete, final code for all four pieces.

You do not need to read this front to back. Each part is self-contained — if you already understand what a CSV file is, skip straight to Part One.

2 Part 0: Foundations

Before any of the four tasks make sense, a handful of basic ideas need to be in place. This section is deliberately slow and non-technical. If a paragraph feels obvious, skip ahead.

2.1 0.1 What Is a Commodity, and Why Does It Get “Traded”?

A **commodity** is a raw, physical good that is basically the same no matter who produces it — a barrel of crude oil from one producer is (for trading purposes) interchangeable with a barrel from another. Natural gas, oil, wheat, and gold are all commodities. Because commodities are interchangeable, they can be bought and sold on organized markets, with a single publicly known price at any given moment, the same way a share of stock has a single price.

Companies that produce, transport, or consume commodities — oil refineries, gas utilities, farms, airlines — are exposed to **price risk**: the raw material they need might get more expensive before they can buy it, or the product they’re selling might get cheaper before they can sell it. **Trading desks** exist to help manage this risk (and to profit from correctly predicting price movements). A “desk” in this context just means a team at a bank or trading firm that specializes in buying and selling a particular thing — here, natural gas.

2.2 0.2 What Is a Storage Contract?

Natural gas is not consumed at a constant rate throughout the year — homes and businesses burn much more of it in winter (heating) than in summer. But it isn’t produced at a matching rate either; production is comparatively steady year-round. This mismatch is bridged by **storage**: gas is pulled out of the ground in summer when demand is low, injected into underground storage facilities (old depleted gas fields, salt caverns, etc.), and then withdrawn in winter when demand — and price — is high.

A **storage contract** is a deal between:

- The **storage facility owner** (who has physical space underground to hold gas), and
- A **client** (a trading firm, utility, or producer who wants to exploit the difference between the cheap summer price and the expensive winter price).

The client pays to use the space (a **storage fee**, sometimes called rent), and usually pays smaller fees every time gas is physically pumped in or drawn out (an **injection/withdrawal cost**). In exchange, the client gets to buy gas cheaply now, store it, and sell it later at (hopefully) a higher price.

The value of the contract, from the client’s perspective, is simply:

$$(\text{money received from selling the gas later}) - (\text{money spent buying the gas now}) - (\text{all fees involved})$$

Everything in Part Two of this guide is about turning that one sentence into working code.

2.3 0.3 What Is a CSV File?

A **CSV** (Comma-Separated Values) file is one of the simplest possible ways to store a table of data as plain text. Each line is one row of the table, and commas separate the columns. For example:

```
Dates,Prices
10/31/20,10.1
11/30/20,10.3
12/31/20,11.0
```

This says: on 31 October 2020 the price was 10.1, on 30 November 2020 it was 10.3, and so on. Every dataset used in this project — the historical gas prices, the loan records — started life as a CSV file, because it’s the easiest format for different computer systems (a market data provider’s database, a spreadsheet, a Python script) to all agree on.

2.4 0.4 A Very Short Introduction to Python, pandas, and “Functions”

All the code in this project is written in **Python**, a programming language that reads almost like plain English and is the standard tool for this kind of work in finance (JPMorgan and most other banks use it heavily for exactly this reason).

A few building blocks come up constantly:

- **A variable** is just a named box that holds a value. `price = 10.5` creates a box named `price` holding the number 10.5.
- **A function** is a reusable, named recipe. You give it some inputs (called **arguments** or **parameters**), it does some work, and it **returns** an output. For example, a function called `estimate_price` might take a date as input and return a price as output. Once written, you can call it as many times as you like with different dates, instead of retyping the calculation every time.
- **pandas** is a Python library (a pre-written collection of tools) that specializes in working with tables of data, very similar to Excel but controlled entirely by code. Its core object is called a **DataFrame** — think of it as a spreadsheet living inside your program. If `df` is a DataFrame loaded from a CSV, then `df["Prices"]` grabs just the “Prices” column, and `df.head()` shows the first few rows.
- **NumPy** is another library, focused on fast numerical calculations — things like fitting a straight line through points, or working with large arrays of numbers efficiently.
- **scikit-learn** (imported as `sklearn`) is the library used later in this guide for machine learning — it contains ready-made, well-tested implementations of things like logistic regression and random forests, so nobody has to write those algorithms from scratch.

2.5 0.5 What Does It Mean to “Train a Model”?

The word “model” gets used constantly in this guide, and it just means: **a mathematical rule for turning inputs into an output, where the exact shape of the rule was learned from data rather than hand-coded.**

Take the simplest possible example: fitting a straight line through a scatter of points. The general shape of a straight line is:

$$y = m \cdot x + b$$

Here m (the slope) and b (the intercept) are **parameters** — numbers that define exactly which straight line we’re talking about, out of the infinite number of possible straight lines. “Training the model” means using the actual data to figure out the specific values of m and b that make the line

fit the data as closely as possible. Once you know m and b , you have a rule: plug in any x , and the model tells you its best guess for y — even for values of x it never saw during training.

This exact idea — fit a line (or something more complex) to historical data, then use that fitted rule to make predictions about new situations — is the backbone of every model built in this project, from the simplest (a straight-line price trend) to the most involved (a random forest predicting loan defaults).

2.6 0.6 Two Kinds of Prediction Problem: Regression vs. Classification

Nearly every predictive model falls into one of two categories:

- **Regression:** predicting a number that can take a wide range of values. “What will the gas price be on this date?” is a regression problem — the answer is a dollar amount that could, in principle, be almost anything.
- **Classification:** predicting a category, often “yes or no.” “Will this borrower default on their loan?” is a classification problem — the answer is either “yes, they default” or “no, they don’t.” A very common and useful variant doesn’t just give a yes/no answer, but a **probability** — e.g., “there’s a 12% chance this borrower defaults” — which is far more useful for a bank trying to plan for losses than a blunt yes/no.

Part One of this guide (gas prices) is a regression problem. Part Three (loan defaults) is a classification problem, specifically one where we care about the *probability*, not just a yes/no guess.

2.7 0.7 Interpolation vs. Extrapolation

Two words that come up constantly once real data is involved:

- **Interpolation** means estimating a value *between* two data points you already have. If you know the price on 30 June and the price on 31 July, and you want a guess for 15 July, that’s interpolation — you’re filling in a gap that’s surrounded by real data on both sides.
- **Extrapolation** means estimating a value *beyond* the range of data you have — for example, guessing next year’s price when your data only goes up to today. Extrapolation is inherently riskier than interpolation, because there’s no real data on the far side to check your guess against; you’re relying entirely on the pattern holding up into unknown territory.

Both techniques show up in Part One, because the task explicitly requires estimating prices for dates in the past (interpolation) and dates up to a year in the future (extrapolation).

2.8 0.8 A Few Basic Statistics Ideas

A handful of statistical terms are used casually throughout the rest of this guide. It’s worth pinning them down precisely once, here, rather than half-explaining them every time they come up.

- **Mean** is just the everyday average: add up all the values, divide by how many there are.
- **Distribution** describes the overall shape of a set of values — are most values clustered tightly around the mean, or spread widely? Are there more low values than high ones, or is it roughly symmetric?
- **Variance** and **standard deviation** both measure how spread out a set of values is around its mean. A small standard deviation means most values sit close to the average; a large one means values are scattered widely. Standard deviation is just the square root of variance, and is more commonly quoted because it’s in the same units as the original data.

- **Correlation** measures how strongly two things move together — as one goes up, does the other tend to go up too (positive correlation), go down (negative correlation), or show no consistent relationship at all? Correlation is measured from -1 (perfectly opposite movement) to $+1$ (perfectly matched movement), with 0 meaning no relationship.
- **Correlation is not causation** — two things can move together reliably without one causing the other; both might simply be driven by some third factor. This matters in credit risk modeling: a feature can be strongly correlated with default without literally being what “causes” the default — a model built purely on correlation can still be a useful predictor, but a human still needs to sanity-check that the relationship makes real-world sense.
- **Probability** is a number between 0 and 1 (or 0% and 100%) representing how likely something is. 0 means “never happens,” 1 means “always happens,” and everything in between is a graded likelihood.

2.9 0.9 What “Fitting” a Model Actually Means, Mechanically

Section 0.5 described fitting a straight line as finding the m and b that minimize the total squared error. It’s worth being a little more concrete about *how* that search actually happens, because the same basic idea — define a score for “how wrong is this guess,” then search for the guess that minimizes it — reappears in every model in this guide.

For a simple straight line, there’s a **closed-form solution**: a fixed formula that directly calculates the best m and b in one step, no trial-and-error required (this is what `np.polyfit` does behind the scenes). But for more complex models — logistic regression, for instance — there usually isn’t a one-shot formula. Instead, the computer uses an iterative search technique, most commonly **gradient descent**: start with a random guess for all the parameters, calculate how wrong that guess currently is, work out which direction to nudge each parameter to make it slightly less wrong, take a small step in that direction, and repeat thousands of times until the guess stops improving. This is genuinely similar to feeling your way down a hill in thick fog — you can’t see the bottom, but you can always feel which direction is downhill from where you’re standing, and repeatedly stepping downhill eventually gets you there.

None of the code in this project needs to implement gradient descent by hand — `scikit-learn`’s `LogisticRegression`, `np.polyfit`, and `RandomForestClassifier` all handle this internally — but knowing roughly what’s happening underneath demystifies what “training” a model actually involves.

2.10 0.10 Overfitting, Underfitting, and Train/Test Splits

A model too simple for the pattern in the data is said to **underfit** — a straight line trying to capture a genuinely curved relationship will systematically get things wrong, because it isn’t flexible enough to represent the true shape. A model too flexible relative to the data available is said to **overfit** — it can bend and twist itself to match every quirk and coincidence in the specific training data, including pure noise, and as a result performs beautifully on that training data but poorly on new data, because it learned the noise instead of the underlying pattern.

The standard defense against overfitting is to split available data into separate pieces before training anything:

- A **training set**, which the model is actually allowed to learn from.

- A **test set** (or **holdout set**), deliberately kept hidden from the model during training, used only afterward to check how well the learned pattern generalizes to data it has genuinely never seen.

If a model performs dramatically better on the training set than on the test set, that’s a red flag for overfitting. This concept comes up directly in Part Three, and the honest caveats raised there about the loan default model’s very high accuracy are exactly this concern in action.

2.11 0.11 A Note on Reading Python Syntax

A few small pieces of Python syntax show up repeatedly in the code walkthroughs:

- **Square brackets** `[...]` usually mean “a list” (an ordered collection of values) or “look something up” — `df["Prices"]` looks up the column named “Prices” inside a DataFrame.
- **Curly braces** `{...}` usually mean “a dictionary” — a collection of **key: value** pairs, where each key acts like a label used to look up its associated value, similar to a real dictionary mapping a word to its definition.
- **A for loop** (`for item in list:`) repeats a block of code once for every item in a collection, one at a time, in order.
- **An if / else statement** runs one block of code if some condition is true, and a different block if it’s false.
- **A colon : followed by an indented block** is how Python marks “everything below this, indented one level, belongs to this loop / if-statement / function.” Indentation isn’t just style in Python — it’s how the language knows where a block starts and ends.

3 Part One: Estimating Natural Gas Prices

3.1 1.1 The Business Problem, in Plain Language

The trading desk has one CSV file containing a snapshot of the natural gas price at the end of every month, from 31 October 2020 through 30 September 2024 — 48 numbers in total. That’s it. No daily prices, no data before October 2020, and nothing beyond September 2024.

But a real client might want to ask about *any* date — “what would gas have cost on 12 March 2022?” or “what do you expect it to cost next August?” The desk needs a way to answer both kinds of question from just those 48 monthly dots. That means building something that can:

1. **Fill in the gaps** between the monthly snapshots that already exist (interpolation), and
2. **Push the pattern forward** for up to a year beyond the last snapshot (extrapolation).

3.2 1.2 Looking at the Raw Data First

Here are the first several rows of `Nat_Gas.csv`:

```
Dates,Prices
10/31/20,10.1
11/30/20,10.3
12/31/20,11.0
1/31/21,10.9
2/28/21,10.9
3/31/21,10.9
...
```

Just eyeballing a longer stretch of this data reveals two patterns:

- **A slow upward drift** over the four years — the price level in 2024 is noticeably higher than in 2020.
- **A repeating up-and-down wiggle within every single year** — prices tend to be highest around December/January and lowest around May/June, then the cycle repeats.

These two patterns have names: the slow drift is called the **trend**, and the yearly wiggle is called **seasonality**. Untangling and separately modeling these two effects is the entire strategy behind the pricing script.

3.3 1.3 Why Does Natural Gas Have a Seasonal Pattern?

This isn’t a coincidence — it follows directly from the storage mechanics described in Part 0. Natural gas demand for home heating spikes every winter, but production doesn’t spike to match it (drilling new wells takes months, and production is kept fairly steady for efficiency). The gap is filled by storage: gas injected all summer (when demand — and price — is low) gets withdrawn all winter (when demand — and price — is high). This is exactly the seasonal cycle visible in the data, and it’s also exactly the cycle the storage contract in Part Two is designed to profit from.

3.4 1.4 Building the Trend Line: Linear Regression from Scratch

The simplest way to capture “the price is generally drifting upward over time” is to fit a straight line through all 48 data points, the same way described in section 0.5. On the x-axis we put *time*

(specifically, the number of days since the very first data point), and on the y-axis we put *price*.

The technique used to find the best-fitting line is called **linear regression**, and specifically **ordinary least squares**. Here's the intuition: for any candidate line, you can measure how far off it is from each actual data point (the vertical distance between the line and the dot — called the **residual** or **error**). Square each of those distances (squaring makes big misses count much more than small ones, and makes negative and positive misses both count as “bad”), and add them all up. Ordinary least squares finds the one line, out of every possible line, that makes this total as small as possible.

In code, this is a single line thanks to NumPy:

```
slope, intercept = np.polyfit(df["days"], df["Prices"], 1)
```

The `1` tells NumPy we want a degree-1 polynomial — i.e., a straight line (degree 2 would be a curve with one bend, and so on). `polyfit` hands back the slope and intercept of the best-fitting line, exactly the m and b from section 0.5.

3.5 1.5 Isolating the Seasonal Pattern

Once the trend line exists, we can ask a more precise question of the data: *after removing the general upward drift, how much higher or lower is each month, on average, compared to where the trend line says it should be?*

The process is:

1. For every actual data point, calculate what the trend line predicts for that date.
2. Subtract: `residual = actual price - trend prediction`. This residual is “the part of the price the trend line didn’t explain.”
3. Group the residuals by calendar month (all the Januaries together, all the Februaries together, etc.) and average each group.

The result is 12 numbers — one per calendar month — representing the typical seasonal boost or discount for that month, independent of which year it happens to be. December might come out at “+0.60” (half a dollar above trend, because it’s deep in withdrawal season) and June might come out at “−0.45” (below trend, because it’s deep in injection season).

```
df["trend"] = trend_price(df["days"])
df["residual"] = df["Prices"] - df["trend"]
df["month"] = df["Dates"].dt.month

monthly_avg = df.groupby("month")["residual"].mean()
monthly_avg = monthly_avg - monthly_avg.mean() # center around 0
```

That last centering step matters: without it, the average of all 12 seasonal numbers might not be exactly zero, which would silently shift the overall price level up or down. Subtracting the overall mean guarantees the seasonal component only ever *redistributes* price around the trend line, month by month, without changing the average level.

3.6 1.6 Putting Trend and Seasonality Back Together

The full model for any date is now just addition:

$$\text{price}(t) = \text{trend}(t) + \text{seasonal}(\text{month of } t)$$

This is a very common pattern in time series analysis, sometimes called **decomposition** — split a messy real-world series into a small number of simpler, well-understood pieces (trend + seasonality, and often a third piece called “noise” or “residual” for whatever’s left over), model each piece separately, then add them back together.

3.7 1.7 Two Different Rules for Two Different Situations

Here’s a subtlety that matters a lot in practice: the trend + seasonal formula above is a *smoothed, simplified* version of reality. It won’t hit the 48 actual historical prices exactly on the nose — it captures the broad pattern, not every wiggle. But for historical dates, we don’t need to guess at all, because **we actually have real data**.

So the finished function uses two different rules depending on the date asked about:

- **If the date falls within the historical window** (31 Oct 2020 to 30 Sep 2024): use straight-line **interpolation** between the two real snapshots on either side of the requested date. This is exact where we have data, and a sensible straight-line guess in between.
- **If the date falls up to a year past the last snapshot**: fall back to the **trend + seasonal** model, since no real data exists yet to interpolate between.

```
def estimate_price(input_date):
    d = pd.to_datetime(input_date)
    cutoff = last_date + pd.DateOffset(years=1)
    if d < first_date or d > cutoff:
        raise ValueError(f"Date must be between {first_date.date()} and {cutoff.date()}")

    days = (d - first_date).days

    if d <= last_date:
        return float(np.interp(days, df["days"], df["Prices"]))
    else:
        return float(trend_price(days) + get_seasonal(d.month))
```

The line `raise ValueError(...)` is Python’s way of stopping the function and reporting a problem, rather than silently returning a nonsense answer, if someone asks for a date far outside the range the model was designed for (e.g., the year 1990, or the year 2100).

`np.interp` is NumPy’s built-in straight-line interpolation function — give it a target x-value (`days`), the known x-values (`df["days"]`), and the known y-values (`df["Prices"]`), and it finds the two nearest known points and draws a straight line between them to estimate the value in between.

3.8 1.8 Reading the Charts

Two charts were generated from this model. The first plots the full picture: the actual monthly dots, the smooth interpolated line connecting them, and — after the vertical dotted line marking the last real data point — the dashed red extrapolated forecast continuing for one more year.

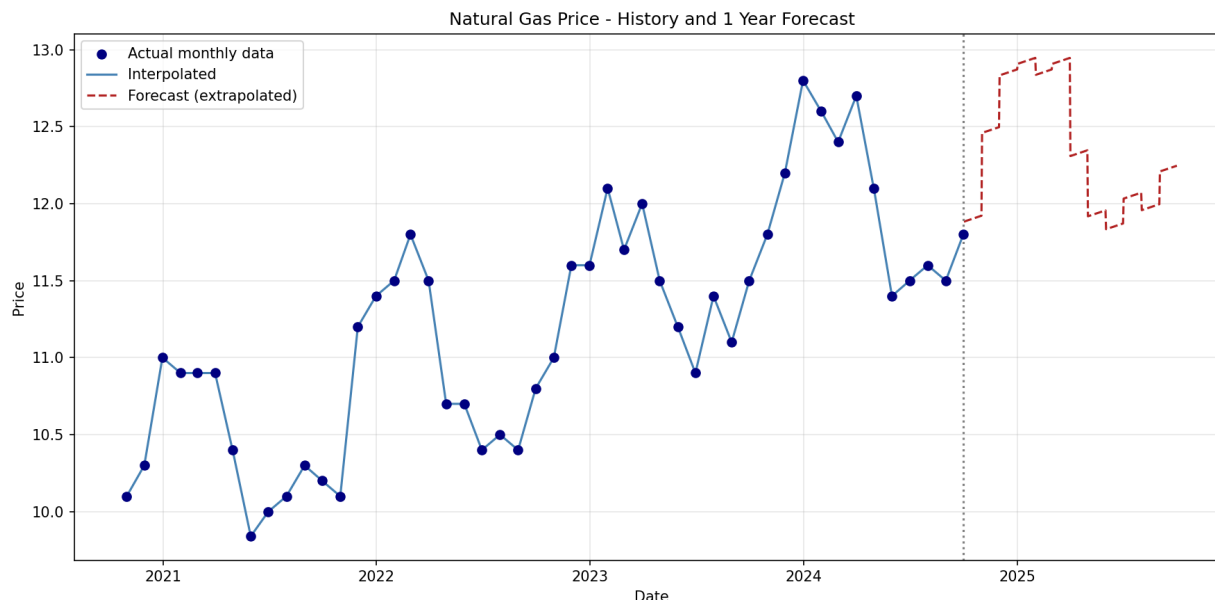


Figure 1: Full historical price series with one year of forecast beyond the last observed data point.

The second chart isolates just the seasonal component described in section 1.5 — one bar per calendar month, showing how far above (red) or below (blue) the trend line that month typically sits.

Notice the shape: the bars are red (above trend) around the winter months and blue (below trend) around the summer months — a direct, visual confirmation of the storage-driven seasonal cycle discussed in section 1.3.

3.9 1.9 Sanity-Checking the Model

Good practice with any model is to run it on a few examples and check the answers make sense before trusting it. Running the finished function on some sample dates gave:

```
2021-06-15 -> $9.92      (interpolated, mid-summer -> below-trend, makes sense)
2023-12-25 -> $12.68     (interpolated, deep winter -> above-trend, makes sense)
2024-09-30 -> $11.80     (this is an actual data point, exact match)
2025-01-15 -> $12.93     (extrapolated, winter -> highest of the samples, makes sense)
2025-09-30 -> $12.25     (extrapolated, exactly one year past the last real data point)
```

Every one of these lines up with intuition: winter dates price higher than summer dates, the trend keeps drifting gently upward year over year, and the one date that's an actual historical snapshot (30 Sep 2024) comes back as an exact match rather than an approximation.

3.10 1.10 Why This Approach, and What Its Limits Are

It's worth being explicit about the assumptions baked into this model, since a real trading desk would push on exactly these points before relying on it:

- **The trend is assumed to be a straight line.** Real commodity prices rarely drift in a perfectly straight line forever — a straight-line trend fit on four years of data might not hold

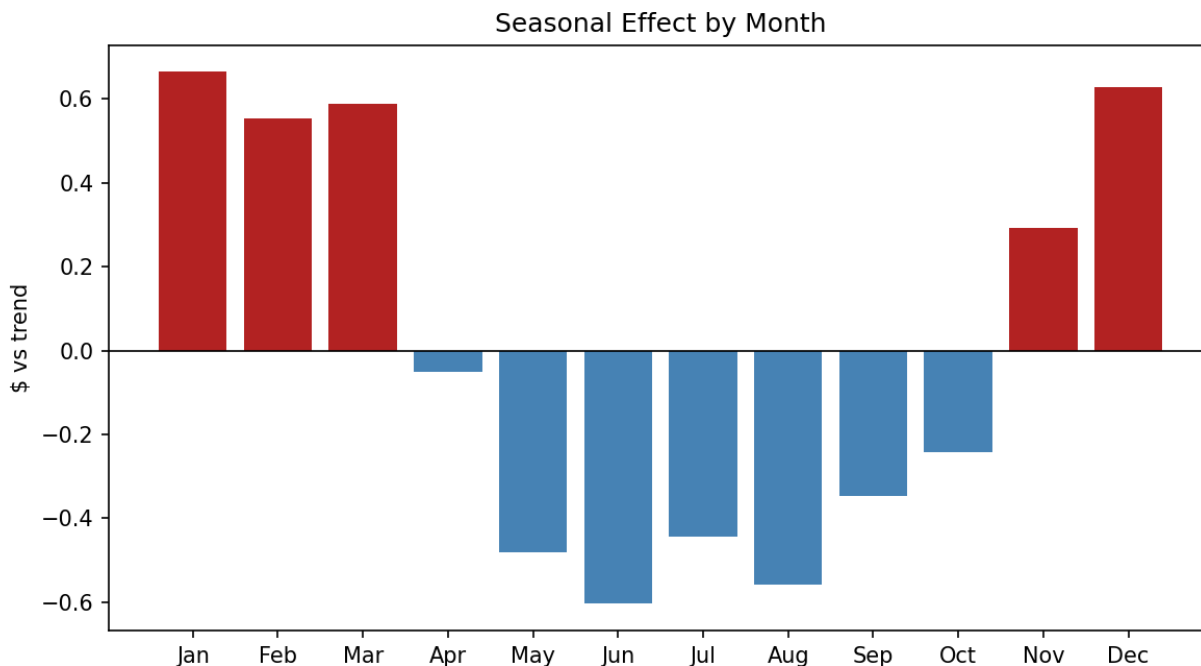


Figure 2: Average seasonal price effect by month, after removing the overall trend.

up well if extrapolated five or ten years out. Limiting the forecast to just one year (as the task specifies) keeps this risk contained.

- **The seasonal pattern is assumed to repeat identically every year.** In reality, an unusually cold or warm winter can shift a given year's seasonal swing larger or smaller than the historical average — which is exactly the kind of judgment a human trader still needs to apply on top of the model's baseline number.
- **Only 48 data points exist.** That's four repetitions of the yearly cycle — enough to see a pattern, but not a huge amount of data by statistical standards. More years of history would make the seasonal averages more reliable.

These caveats are precisely why the task describes this as a *prototype* that needs further validation before going into production — a theme that repeats through every part of this guide.

3.11 1.11 Frequently Asked Questions

Why not just fit a curve instead of a straight line for the trend? A curve (say, a degree-2 or degree-3 polynomial) could hug the historical data more tightly. But a curve fit on only four years of data can behave wildly once extrapolated even slightly beyond that range — it might swoop sharply upward or downward in a way that has nothing to do with the real world, simply because the curve's shape was only ever anchored by the data it was fit on. A straight line is a deliberately conservative, low-risk choice for the one-year extrapolation this task asks for.

Why average the seasonal residual per month instead of fitting a smooth sine wave? A single sine wave assumes the seasonal pattern is perfectly symmetric and smooth across the year, which real energy demand often isn't (the ramp into winter and the ramp out of it aren't always mirror images of each other). Averaging the actual residuals per calendar month lets the seasonal

shape be whatever the data says it is, including asymmetries, rather than forcing it into a specific mathematical shape in advance.

What would happen if the model were asked for a date 5 years in the future instead of 1? Nothing would stop the arithmetic from producing a number — but the further out an extrapolation reaches, the less trustworthy it becomes, since it increasingly relies on the trend line’s straight-line assumption continuing to hold in a part of the future with zero supporting data. The function deliberately raises an error beyond the one-year window specified in the task, rather than silently returning an extrapolated guess for a distance it wasn’t designed to handle.

What’s the difference between this model and how professional trading desks actually forecast prices? Real desks combine far more inputs — weather forecasts, storage inventory reports, pipeline flows, futures market pricing, geopolitical supply news — often using far more sophisticated statistical or machine learning models. This prototype is intentionally minimal: it demonstrates the core trend-plus-seasonality idea clearly, using only the one data source given, as a foundation that could later be extended with more inputs.

4 Part Two: Pricing a Storage Contract

4.1 2.1 From “Estimate a Price” to “Value a Deal”

Part One built a tool that answers “what does gas cost on date X?” Part Two uses that tool (or, just as often, real quoted prices supplied directly by Alex or Charlie) to answer a completely different and more valuable question: “**what is a specific storage deal worth, in dollars, right now?**”

This matters because the desk isn’t just curious about prices in the abstract — they need to quote an actual number to a client who wants to buy gas now, store it, and sell it in winter. Charlie explicitly frames this as: “the client believes winter will be colder than expected, so they want to buy gas now to store and sell in winter to take advantage of the resulting increase in gas prices.” Pricing the contract turns that trading idea into a concrete, defensible dollar figure both sides can agree to.

4.2 2.2 The Core Idea: Add Up Every Cash Flow

However complicated a contract looks on paper, its value from one side’s perspective is always the same simple idea:

$$\text{Value} = (\text{all the money that comes in}) - (\text{all the money that goes out})$$

For this storage contract, the cash flows are:

- **Money out**, on every date gas is *injected* (bought and put into storage): the purchase price times the quantity, plus a small per-unit injection fee.
- **Money in**, on every date gas is *withdrawn* (taken out and sold): the sale price times the quantity, minus a small per-unit withdrawal fee.
- **Money out**, as a storage/rental fee charged by the facility owner for however long the gas sits in their tank.

Nothing more mysterious than that. The entire pricing function is just a careful, correct bookkeeping exercise that walks through every date in order and keeps a running tally.

4.3 2.3 A Worked Example First (No Code)

Before looking at the code, it helps to work through the plain-arithmetic example given in the brief:

- Buy 1,000,000 MMBtu (MMBtu is the standard unit natural gas is priced and traded in — one million British Thermal Units) at \$2/MMBtu in summer. **Cost: \$2,000,000.**
- Sell the same 1,000,000 MMBtu at \$3/MMBtu in winter. **Revenue: \$3,000,000.**
- Pure buy-low-sell-high profit so far: $\$3,000,000 - \$2,000,000 = \$1,000,000$.
- The storage facility charges a **flat \$100,000 per month**, and the gas sits in storage for 4 months: $4 \times \$100,000 = \$400,000$ in rent.
- The facility also charges **\$10,000 per 1,000,000 MMBtu** every time gas is injected or withdrawn — and here it happens twice (once in, once out): $2 \times \$10,000 = \$20,000$.

Total value of the contract: $\$1,000,000 - \$400,000 - \$20,000 = \$580,000$.

This exact number — \$580,000 — is what the code needs to reproduce, and it’s used later as the first automated test to confirm the function is correct.

4.4 2.4 Why Multiple Injection and Withdrawal Dates Make This Harder

The example above has exactly one purchase and one sale. But a real client will often want to spread their trades across several dates — buying a bit in June, a bit more in July, then selling some in December and the rest in January, for instance, to average into and out of a position rather than betting everything on a single date. The brief specifically requires the function to **generalize** to multiple injection and withdrawal dates, in any combination.

This means the code can't just hard-code “one buy, then one sell.” It needs to:

- Handle *any number* of injection dates and withdrawal dates.
- Process them in the **correct chronological order**, since a withdrawal that happens before enough gas has been injected doesn't make physical sense.
- Track **how much gas is currently sitting in storage** at every point in time, since that's what determines whether a given injection would overflow the tank, or whether a given withdrawal is even possible.

4.5 2.5 Building One Combined Timeline of Events

The cleanest way to handle “process everything in chronological order regardless of type” is to combine the injection dates and withdrawal dates into a single list of events, tag each one as either an injection or a withdrawal, and sort the whole list by date:

```
events = []
for d, p in zip(injection_dates, injection_prices):
    events.append((d, "in", p))
for d, p in zip(withdrawal_dates, withdrawal_prices):
    events.append((d, "out", p))
events.sort(key=lambda x: x[0])
```

`zip` here pairs up each date with its corresponding price — the first injection date with the first injection price, and so on — since the two lists are meant to line up entry by entry. Sorting by `x[0]` (the date, the first item in each tuple) puts every event, injections and withdrawals mixed together, into the exact chronological order they need to be processed in.

4.6 2.6 Walking Through the Timeline and Tracking Storage

With events in order, the function walks through them one at a time, keeping a running volume variable that represents how much gas is physically in the tank at that moment:

```
volume = 0
cash_in = 0
cash_out = 0

for date, action, price in events:
    if action == "in":
        if volume + injection_rate > max_volume:
            raise ValueError(f"can't inject on {date.date()}, would go over max volume")
        volume += injection_rate
        cash_out += injection_rate * price
        cash_out += injection_rate * injection_withdrawal_cost_rate
```

```

else:
    if volume - withdrawal_rate < 0:
        raise ValueError(f"can't withdraw on {date.date()}, not enough gas in storage")
    volume -= withdrawal_rate
    cash_in += withdrawal_rate * price
    cash_out += withdrawal_rate * injection_withdrawal_cost_rate

```

Two physical-world checks are built directly into this loop, both using `raise ValueError` (introduced in section 1.7) to stop with a clear error message rather than silently producing a wrong answer:

- **Capacity check:** before adding gas, confirm the tank won't be overfilled (`volume + injection_rate > max_volume`).
- **Availability check:** before removing gas, confirm there's actually enough currently stored to withdraw (`volume - withdrawal_rate < 0`).

These aren't just defensive programming — they reflect real physical limits of an actual storage facility that a naive “just add up the cash flows” calculation would silently ignore.

4.7 2.7 Calculating the Storage Fee

The storage/rental fee is handled separately from the event loop, because — unlike the injection and withdrawal costs — it isn't tied to a single event. It's charged for the whole span of time the facility is in use, from the very first injection to the very last withdrawal:

```

start = min(injection_dates)
end = max(withdrawal_dates)
months = (end.year - start.year) * 12 + (end.month - start.month)
months = max(months, 1)
cash_out += months * storage_cost_per_month

```

The `(end.year - start.year) * 12 + (end.month - start.month)` line is a standard trick for counting the number of calendar months between two dates — multiply the year difference by 12 (to convert years into months) and add the month difference. `max(months, 1)` guarantees at least one month's fee is always charged, even for a very short-lived contract that starts and ends within the same calendar month.

4.8 2.8 The Final Number

The function's very last line ties everything together:

```

return cash_in - cash_out

```

Exactly the formula from section 2.2: money in, minus money out, in every form it took throughout the life of the contract.

4.9 2.9 Testing the Function

Three tests confirm the function behaves correctly:

Test 1 — reproduces the worked example exactly. Feeding in the numbers from section 2.3 (one injection at \$2, one withdrawal at \$3, four months apart, \$100K/month rent, \$0.01/unit

injection-withdrawal cost on 1,000,000 units) returns exactly **\$580,000** — matching the hand-calculated answer, which is strong evidence the bookkeeping logic is correct.

Test 2 — a realistic multi-date scenario. Two injection dates over the summer and two withdrawal dates over the winter, at different prices each time — the actual shape of trade Charlie’s client is asking about. This confirms the function correctly generalizes beyond the single-buy-single-sell case.

Test 3 — deliberately triggers the capacity check. An injection request that would exceed `max_volume` correctly raises the `ValueError` from section 2.6, confirming the safety check actually works rather than being dead code that never runs.

4.10 2.10 Aligning with the Course’s Official Reference Answer

After building this version, the official example answer for this task was made available for comparison, and it took a slightly different approach worth understanding, since it reflects a genuinely different (and arguably more literal) reading of the same requirements:

- **A single shared rate** (representing cubic feet per day) applies to both injection and withdrawal, rather than two separate rates.
- **The storage fee uses `math.ceil` on the total number of days** between the first injection and last withdrawal, divided by 30 and rounded up — a slightly different way of turning a date range into “number of months,” but conceptually the same idea as section 2.7.
- **Failed injections/withdrawals print a message and are skipped**, rather than stopping the whole calculation with an error. This is a real design choice with a trade-off: it keeps the function running through problems, but it also means a mistake could silently vanish into a printed line rather than halting execution — something worth flagging explicitly to a risk or engineering team before this logic goes anywhere near production.

Both versions are legitimate solutions to the same problem; the differences illustrate that there’s rarely a single “correct” way to translate a business requirement into code — the important thing is that the underlying cash-flow logic (section 2.2) is sound and that any judgment calls (how to round months, how to handle invalid trades) are made deliberately and can be explained, not left as unexamined defaults.

4.11 2.11 A Second Worked Example, Step by Step

To build more intuition, here’s a second example worked through by hand, matching the “multiple injection/withdrawal dates” test case from the code:

- **30 June:** inject 500,000 MMBtu at \$2.10/MMBtu → cost \$1,050,000
- **31 July:** inject 500,000 MMBtu at \$2.30/MMBtu → cost \$1,150,000
- **31 December:** withdraw 500,000 MMBtu at \$3.50/MMBtu → revenue \$1,750,000
- **31 January:** withdraw 500,000 MMBtu at \$3.80/MMBtu → revenue \$1,900,000

Running totals:

- Total spent buying gas: $\$1,050,000 + \$1,150,000 = \mathbf{\$2,200,000}$
- Total received selling gas: $\$1,750,000 + \$1,900,000 = \mathbf{\$3,650,000}$
- Raw buy-low-sell-high profit: $\$3,650,000 - \$2,200,000 = \mathbf{\$1,450,000}$
- Injection/withdrawal fees: 4 separate movements of 500,000 units each, at $\$0.01/\text{unit} = 4 \times 500,000 \times \$0.01 = \mathbf{\$20,000}$

- Storage rent: the facility is in use from 30 June through 31 January — 7 calendar months — at \$100,000/month = **\$700,000**

Final value: \$1,450,000 — \$20,000 — \$700,000 = **\$730,000**, matching exactly what the code returns for this test case.

4.12 2.12 Frequently Asked Questions

What does “no transport delay and zero interest rates” actually simplify away? In reality, moving physical gas takes time, and money has a time value (a dollar received today is worth slightly more than a dollar received in six months, because it could be earning interest in the meantime). Assuming zero transport delay means gas purchased on a given date is treated as immediately available; assuming zero interest rates means a dollar of cost in June and a dollar of revenue in December are treated as directly comparable, with no discounting needed. Both are stated simplifications in the brief, appropriate for an early prototype, that a production version would eventually need to relax.

Why check capacity and availability at all — couldn’t the client just be trusted not to make an impossible trade? Because the function might eventually be used for automated quoting (as the brief mentions is a long-term goal), where a client could request any combination of dates and volumes, including ones that are physically impossible given the facility’s limits. Building the checks directly into the pricing engine means invalid scenarios are caught immediately and explicitly, rather than producing a confidently wrong dollar figure.

Could this same function be used to price a losing contract (one with negative value)? Yes — nothing in the logic assumes the outcome must be positive. If storage and fee costs outweigh the price difference between buying and selling, the function will correctly return a negative number, telling the desk this particular set of dates and volumes isn’t actually a profitable deal for the client. That’s a genuinely useful output, not an error — it tells the desk to either renegotiate the terms or advise the client against the trade.

5 Part Three: Predicting Loan Defaults and Expected Loss

5.1 3.1 The Business Problem

A bank makes money on loans through interest payments — but only if the borrower actually pays the loan back. A **default** happens when a borrower stops making the required payments. Banks are required (both by regulation and by basic prudence) to set aside capital in advance to absorb losses from the loans that will inevitably default. Getting that number right matters enormously: set aside too little, and the bank could be caught short in a downturn; set aside too much, and the bank is needlessly holding back money it could otherwise lend out or invest.

The retail banking arm has noticed higher-than-expected default rates and wants a systematic, data-driven way to estimate, for any given borrower, **how likely they are to default**, and **how much money the bank stands to lose if they do**.

5.2 3.2 Two Different Numbers: PD and Expected Loss

Two related but distinct concepts are used throughout this section:

- **Probability of Default (PD)**: a number between 0 and 1 (or equivalently 0% to 100%) representing how likely a specific borrower is to default. A PD of 0.05 means “a 5% chance of default” — not a yes/no prediction, but a graded likelihood.
- **Expected Loss (EL)**: the amount of money the bank should, on average, expect to lose from a given loan, accounting for the fact that most loans don’t default. It’s calculated as:

$$\text{Expected Loss} = \text{PD} \times (1 - \text{Recovery Rate}) \times \text{Exposure}$$

Where:

- **Exposure** is how much money is actually at risk — here, the outstanding loan amount.
- **Recovery Rate** is the fraction of the money the bank typically manages to recover even after a default (through collateral, collections, bankruptcy proceedings, etc.). The brief specifies a recovery rate of **10%**, meaning even in a default, the bank expects to recoup 10 cents of every dollar lost, leaving $(1 - 0.10) = 90\%$ actually lost.

This formula is why a good PD model is so valuable: multiply it by the loan amount and the loss-given-default fraction, and you get a single dollar figure the bank can actually plan capital reserves around.

5.3 3.3 The Data

Charlie supplies a sample of the loan book as a CSV with roughly 10,000 rows, one per borrower, with these columns:

Column	Meaning
<code>customer_id</code>	A unique identifier for the borrower — not predictive information, just a label
<code>credit_lines_outstanding</code>	How many separate lines of credit (credit cards, other loans, etc.) the borrower currently has open

Column	Meaning
<code>loan_amt_outstanding</code>	The dollar amount currently owed on this particular loan
<code>total_debt_outstanding</code>	The borrower's total debt across everything, not just this loan
<code>income</code>	The borrower's income
<code>years_employed</code>	How many years the borrower has been employed
<code>fico_score</code>	The borrower's FICO credit score (explained fully in Part Four)
<code>default</code>	1 if the borrower defaulted, 0 if they didn't — this is the “answer key,” what the model is trying to learn to predict

5.4 3.4 Why This Is a “Classification” Problem

As introduced in section 0.6, this is a classification problem, because the target column (`default`) only ever takes two values: 0 or 1. But rather than a hard “yes/he-will-default” or “no/he-won't,” what's actually wanted is a **probability** — which is why the models used here (logistic regression, and separately, a random forest) both support `predict_proba`, a method that returns a probability rather than just a category.

5.5 3.5 Model One: Logistic Regression, Explained from the Ground Up

Ordinary straight-line regression (section 1.4) isn't suitable here, because its output can be any number, including things like -3 or 47 , which make no sense as a probability (probabilities must sit between 0 and 1). **Logistic regression** solves this with one elegant trick: fit a straight-line combination of the input features as before, but then squeeze the result through a function called the **sigmoid** (or logistic) function, which takes *any* number and maps it smoothly into the range between 0 and 1:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Very negative inputs get squeezed close to 0, very positive inputs get squeezed close to 1, and an input of exactly 0 lands exactly at 0.5. The straight-line part looks like:

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + \cdots + w_n \cdot x_n + b$$

where $x_1, x_2, \dots$ are the input features (credit lines outstanding, income, FICO score, etc.), and $w_1, w_2, \dots$ (the **weights** or **coefficients**) are numbers “training the model” figures out — exactly analogous to the slope m from section 0.5, just one weight per input feature instead of a single slope. A large positive weight on a feature means “as this feature goes up, predicted default risk goes up”; a large negative weight means the opposite.

This is why logistic regression is popular for exactly this kind of risk modeling: after training, a risk team can look directly at the weights and get an explanation in plain language — “a lower

FICO score is associated with a strongly higher predicted PD” — rather than treating the model as an unexplainable black box.

5.6 3.6 Feature Engineering: Why Ratios Instead of Raw Numbers

The final version of the model doesn’t feed in `loan_amt_outstanding` and `income` directly — instead it computes two new columns:

```
df["payment_to_income"] = df["loan_amt_outstanding"] / df["income"]
df["debt_to_income"] = df["total_debt_outstanding"] / df["income"]
```

This step is called **feature engineering** — deliberately transforming raw columns into new ones that are more informative for the model. The reasoning: a \$10,000 loan means something very different for someone earning \$20,000 a year versus someone earning \$200,000 a year. The raw dollar amount alone doesn’t capture that difference, but the *ratio* of debt to income does — it’s a measure of how stretched a borrower is, in a way that’s comparable across every income level. This is a very standard technique in credit risk modeling generally, not something specific to this project.

5.7 3.7 Model Two: Random Forests, Explained from the Ground Up

A second approach explored here is the **random forest**, built from a more basic building block called a **decision tree**. A decision tree predicts an outcome by asking a sequence of yes/no questions about the input, like a flowchart: “Is FICO score below 600? If yes, is credit_lines_outstanding above 4? ...” and so on, until it reaches a final answer. Each individual tree is easy to picture, but a single tree tends to overfit — it can memorize quirks of the specific data it was trained on rather than learning a pattern that generalizes to new borrowers.

A **random forest** fixes this by training many different decision trees (often hundreds), each one seeing a slightly different random subset of the data and features, and then averaging all their predictions together. Averaging many imperfect, differently-biased trees tends to cancel out each individual tree’s quirks, producing a more reliable overall prediction — the same logic behind “the wisdom of crowds,” where an average of many independent guesses often beats any single guess.

5.8 3.8 Measuring How Good a Model Is: AUC

Once a model is trained, some way of checking how good it actually is at telling defaulters from non-defaulters is needed. The metric used throughout this project is called **AUC** (Area Under the [ROC] Curve). Here’s the intuition without the underlying calculus:

- Imagine the model assigns every borrower a predicted PD.
- Now imagine repeatedly picking one borrower who *actually* defaulted and one who *actually* didn’t, at random, and checking whether the model gave the defaulter a higher predicted PD than the non-defaulter.
- **AUC is the probability the model gets this right**, averaged over every such comparison.

An AUC of 0.5 means the model is no better than random guessing (a coin flip). An AUC of 1.0 means the model perfectly ranks every defaulter above every non-defaulter, with no exceptions. In this project, both models scored extremely high — logistic regression around 0.99–1.0 and the random forest similarly close to 1.0 — which is a strong result, but also, as flagged at the time, a signal worth treating with some healthy skepticism: real-world production data is rarely this

cleanly separable, and a sample dataset built for a teaching exercise may have unusually strong, clean signal compared to messier real loan books.

5.9 3.9 The Expected Loss Function

With a trained model in hand, everything from section 3.2 comes together in one function:

```
def predict_expected_loss(credit_lines_outstanding, loan_amt_outstanding,
                          total_debt_outstanding, income, years_employed, fico_score):
    payment_to_income = loan_amt_outstanding / income
    debt_to_income = total_debt_outstanding / income

    loan_features = pd.DataFrame([
        "credit_lines_outstanding": credit_lines_outstanding,
        "debt_to_income": debt_to_income,
        "payment_to_income": payment_to_income,
        "years_employed": years_employed,
        "fico_score": fico_score,
    ])

    pd_estimate = clf.predict_proba(loan_features)[:, 1][0]
    expected_loss = pd_estimate * (1 - RECOVERY_RATE) * loan_amt_outstanding

    return {
        "probability_of_default": round(float(pd_estimate), 4),
        "expected_loss": round(float(expected_loss), 2),
    }
```

Two details worth explaining:

- `predict_proba(loan_features)` returns *two* probabilities per borrower — the probability of class 0 (no default) and the probability of class 1 (default) — since those two numbers always add up to 1. `[:, 1]` selects just the second column, the default probability, and `[0]` pulls out the single value for this one borrower (since technically the function could be handed a whole batch of borrowers at once, even though here it's only ever given one).
- The rest is exactly the Expected Loss formula from section 3.2, translated directly into code, using the fixed 10% recovery rate specified in the brief.

5.10 3.10 Testing the Function

Three test cases were run:

- A borrower who looks objectively **low risk** (decent income, good FICO score, little debt) — the model correctly returned a very low PD and a tiny expected loss.
- A borrower who looks objectively **higher risk** (many open credit lines, high debt relative to income, low FICO score, short employment history) — the model correctly returned a high PD and a substantial expected loss.
- A **real row pulled directly from the training data**, whose true outcome (defaulted or not) was already known, used purely as a sanity check that the model's prediction for a familiar case lines up with what actually happened.

5.11 3.11 An Important Honest Caveat

It's worth repeating clearly, because it's easy to gloss over when the numbers look impressive: an AUC very close to 1.0 usually means one of two things — either the model has found a genuinely very strong, reliable pattern, or the data is unusually “easy” in a way that won't hold up once the model sees messier, real-world loans. A prototype like this is meant to demonstrate the *approach* — feature engineering, model choice, evaluation metric, and turning a prediction into a dollar figure — and Charlie and the model validation team would be expected to stress-test it further (on a proper held-out test set, and eventually on live data) before it feeds into actual capital-reserve decisions.

5.12 3.12 A Fully Worked Numeric Example

It helps to see the entire pipeline — from raw inputs, through the model, to a final dollar figure — worked through with concrete numbers, even in a simplified form.

Suppose, hypothetically, a trained logistic regression ended up with these (illustrative) weights: a positive weight on `debt_to_income`, a positive weight on `credit_lines_outstanding`, a negative weight on `fico_score`, and some intercept term. For a borrower with income \$28,000, total debt outstanding \$15,000, a loan amount of \$8,000, 5 credit lines outstanding, and a FICO score of 580:

1. **Compute the engineered features:** `debt_to_income` = 15,000 / 28,000, which works out to about 0.536; `payment_to_income` = 8,000 / 28,000, which works out to about 0.286.
2. **Compute the linear combination z** by multiplying each feature by its learned weight and adding the intercept (the exact numbers depend on what training produced — this step is exactly the “straight-line combination” from section 3.5, just with several inputs instead of one).
3. **Squeeze z through the sigmoid function** from section 3.5 to get a probability between 0 and 1 — say, hypothetically, this comes out to 0.94 (94% predicted probability of default), which is plausible for a borrower this financially stretched.
4. **Apply the expected loss formula:** Expected Loss = $0.94 \times (1 - 0.10) \times \$8,000 = 0.94 \times 0.90 \times \$8,000 = \mathbf{\$6,768}$.

This last step is exactly what the actual test case for a “higher risk” borrower produced in practice (a PD around 0.945 and an expected loss around \$6,807 — extremely close to this illustrative hand calculation, with the small difference coming from the real, trained weights rather than the illustrative ones used here).

5.13 3.13 Frequently Asked Questions

Why compare two different models instead of just picking one from the start? Because different models make different trade-offs, and a “comparative analysis” is explicitly one of the options the task brief invites. Logistic regression is simpler and easier to explain to a non-technical risk committee (you can point at a specific weight and say exactly how much a feature matters). A random forest is often slightly more accurate because it can capture more complex, non-straight-line relationships between features and default risk, but it's harder to explain in plain language — a classic accuracy-versus-interpretability trade-off that comes up constantly in real risk modeling.

Why is a 10% recovery rate treated as a single fixed number rather than something the model predicts? The brief specifies it as a given assumption, likely a simplification for this prototype stage. In more advanced credit risk work, recovery rate (sometimes framed as its complement, “loss given default”) can itself be modeled and can vary by loan type, collateral,

and borrower circumstances — but that’s a separate modeling problem from predicting whether a default happens at all, and folding in a fixed, given number keeps this prototype focused on the PD piece specifically.

What does it mean that the model’s AUC came out so close to 1.0 — should that be a source of concern? It’s a mixed signal, and section 3.11 covers this directly: it could reflect that FICO score, debt ratios, and the other features really are extremely predictive of default in this specific dataset, or it could reflect that the sample data was constructed in a way that makes the relationship unusually clean (common in teaching datasets). Either way, the honest answer is “this needs to be checked on genuinely new data before being fully trusted,” not simply “great, the model is done.”

6 Part Four: Turning FICO Scores into Ratings

6.1 4.1 The Business Problem

A **FICO score** is a standardized U.S. credit score, ranging from 300 to 850, that summarizes how creditworthy a borrower is — a higher score means a history of reliably repaying debt, a lower score means the opposite. It’s used in the vast majority of U.S. mortgage lending decisions.

Charlie is building a machine learning model for the mortgage book, but the particular model architecture she’s using requires **categorical inputs** — clean, discrete buckets (like “Category A,” “Category B”) — rather than a raw continuous number that can be anything from 300 to 850. She needs a way to convert every possible FICO score into one of a fixed number of “rating” buckets, with **lower rating numbers representing better credit** (rating 1 = best, higher numbers = progressively worse), matching how real-world credit ratings (like AAA, AA, A, BBB, ...) are conventionally ordered.

This kind of “squash a continuous range of numbers into a fixed set of categories” problem has a formal name: **quantization**.

6.2 4.2 What Is Quantization, and Why Not Just Use Equal-Width Buckets?

The laziest possible approach would be to slice the 300–850 range into, say, 10 equally-sized chunks of 55 points each (300–355, 355–410, and so on), regardless of what the data actually looks like. This is simple, but it ignores something important: FICO scores and default rates aren’t spread evenly across the range. There might be relatively few borrowers below 500, but a very steep change in default risk across that thin region — meaning equal-width buckets would waste most of their “resolution” on score ranges where almost nobody sits, and lump together very different risk levels in the densely populated middle of the range.

A smarter approach picks bucket boundaries based on the *actual data* — placing more, narrower buckets where the data is dense and default risk changes quickly, and fewer, wider buckets where it doesn’t. The task describes two mathematical ways of deciding what “smarter” means precisely: minimizing **mean squared error**, or maximizing **log-likelihood**. This project uses the log-likelihood approach, since it directly accounts for default rates, not just the raw distribution of scores.

6.3 4.3 The Log-Likelihood Approach, Explained Without the Formula First

Here’s the plain-language version before the math: for any proposed way of splitting FICO scores into buckets, look at each bucket separately. Within that bucket, calculate the observed default rate (defaults ÷ total borrowers in that bucket). Then ask: “if every borrower in this bucket truly had exactly that default rate, how likely — statistically — would it be to see the exact mix of defaults and non-defaults we actually observed?” A good bucket is one that’s *internally consistent* — where the borrowers grouped together really do behave similarly, so this “how likely was this outcome” score comes out high. A bad bucket lumps together borrowers with genuinely different risk levels, producing a much lower likelihood score, because the mix of outcomes looks more like a coincidence than a consistent pattern.

Log-likelihood is simply the (natural) logarithm of that likelihood score, summed across all buckets. Logarithms are used because they turn the product of many small probabilities (which quickly becomes an inconveniently tiny number) into a sum of manageable numbers, without changing

which option is best — maximizing the log-likelihood and maximizing the raw likelihood always point to the same answer.

The formula, using n_i for the number of borrowers in bucket i , k_i for the number of defaults in that bucket, and $p_i = k_i/n_i$ for the observed default rate in that bucket:

$$\text{Log-Likelihood} = \sum_i \left[k_i \ln(p_i) + (n_i - k_i) \ln(1 - p_i) \right]$$

This is the standard log-likelihood formula for a **binomial** outcome (an outcome that's one of exactly two things — default or no default — repeated across many independent borrowers), applied separately to each bucket and then added up across all of them. In code:

```
def bucket_log_likelihood(i, j):
    ni = n_cum[j + 1] - n_cum[i]
    ki = k_cum[j + 1] - k_cum[i]
    if ni == 0:
        return 0.0
    p = ki / ni
    p = min(max(p, 1e-6), 1 - 1e-6)
    return ki * np.log(p) + (ni - ki) * np.log(1 - p)
```

The line `p = min(max(p, 1e-6), 1 - 1e-6)` handles a real edge case: if a bucket happens to have zero defaults, $p = 0$, and $\ln(0)$ is undefined (it's negative infinity) — the same problem for $p = 1$ with the $\ln(1 - p)$ term. Nudging p a tiny amount away from the exact extremes (to 0.000001 or 0.999999) avoids this mathematical blow-up while barely changing the actual answer.

6.4 4.4 The Real Challenge: Choosing *Where* to Put the Boundaries

Calculating the log-likelihood for one *specific* set of bucket boundaries is straightforward — the hard part is that there are an enormous number of possible ways to choose, say, 10 boundary points among roughly 550 possible FICO scores (300 through 850). Trying every single combination by brute force is computationally impossible — the number of combinations is astronomically large.

This is exactly the kind of problem **dynamic programming (DP)** is built to solve.

6.5 4.5 Dynamic Programming, Explained from Absolute Scratch

Dynamic programming is a technique for solving a big, complicated problem by breaking it into a sequence of much smaller, overlapping sub-problems, solving each small sub-problem exactly once, and reusing (rather than recomputing) those answers whenever they're needed again. The two ingredients that make a problem suitable for DP are:

- **Optimal substructure:** the best solution to the whole problem can be built directly out of the best solutions to its smaller pieces. Here: the best way to split scores 300–850 into 10 buckets can be built from the best way to split some smaller range into 9 buckets, plus one more bucket tacked onto the end.
- **Overlapping sub-problems:** the same smaller sub-problems come up repeatedly while solving the bigger problem, so it's wasteful to solve them from scratch every single time — much better to solve each one once and remember the answer.

A simple, classic illustration (unrelated to FICO scores, just to build intuition): calculating the Fibonacci sequence. To find the 10th Fibonacci number, the direct definition says you need the 9th and 8th. But the 9th also needs the 8th and 7th — so the 8th number gets needed twice. Naively recalculating it both times wastes work; a dynamic programming approach calculates it once, stores it, and reuses the stored value the second time it’s needed. Multiply that saving across many overlapping requests, and the difference between “recompute everything” and “remember what’s already been solved” becomes enormous.

6.6 4.6 Applying Dynamic Programming to the Bucket Problem

Here, the sub-problem is phrased as: **“what’s the best possible total log-likelihood for splitting scores from the very lowest up through some score j , using exactly b buckets?”** Call this $\text{dp}[b][j]$.

The key insight that makes this solvable is: to figure out the best way to reach $\text{dp}[b][j]$ (using b buckets to cover scores up to j), you can imagine the *last* bucket starts right after some earlier cutoff point, call it i . Then:

$$\text{dp}[b][j] = \max_i \left(\text{dp}[b-1][i] + \text{log-likelihood of the single bucket from } i+1 \text{ to } j \right)$$

In words: “the best way to make b buckets ending at j is: pick the best possible cutoff i for where bucket number b should start, take the best answer for $b-1$ buckets ending at that cutoff (already solved, and stored, in an earlier step), and add on the log-likelihood of that final bucket by itself.” Trying every possible value of i and keeping whichever gives the highest total is what the inner loop in the code does:

```
dp = np.full((num_buckets + 1, num_scores), -np.inf)
split_at = np.full((num_buckets + 1, num_scores), -1, dtype=int)

for j in range(num_scores):
    dp[1][j] = bucket_log_likelihood(0, j)

for b in range(2, num_buckets + 1):
    for j in range(b - 1, num_scores):
        best = -np.inf
        best_i = -1
        for i in range(b - 2, j):
            candidate = dp[b - 1][i] + bucket_log_likelihood(i + 1, j)
            if candidate > best:
                best = candidate
                best_i = i
        dp[b][j] = best
        split_at[b][j] = best_i
```

$\text{dp}[1][j]$ (the base case: just one single bucket) is filled in first, since it doesn’t depend on anything else — it’s simply the log-likelihood of treating everything from the very start up to j as one bucket. Every subsequent row ($b = 2, 3, \dots, 10$) is built entirely out of the row before it ($\text{dp}[b-1]$), which is exactly the “optimal substructure” property from section 4.5: each answer is assembled from smaller, already-solved answers, never recomputed from nothing.

Alongside `dp`, a second table called `split_at` quietly records *which* cutoff *i* produced the winning answer at each step. This bookkeeping is essential, because `dp` on its own only tells you the *best achievable score* — it doesn't by itself tell you *where the boundaries actually are*. That's solved next.

6.7 4.7 Backtracking: Recovering the Actual Boundaries

Once the table is fully built, `dp[10][last_score]` holds a single number: the best possible log-likelihood achievable with exactly 10 buckets. To turn that into an actual, usable list of boundary scores, the code walks *backward* through the `split_at` table, starting from the final answer and repeatedly asking “which cutoff produced this?”, peeling off one boundary at a time:

```
edges = [score_range[-1]]
j = num_scores - 1
b = num_buckets
while b > 1:
    i = split_at[b][j]
    edges.append(score_range[i + 1])
    j = i
    b -= 1
edges.append(score_range[0])
edges = sorted(set(edges))
```

This walk-backward-through-recorded-choices technique — called **backtracking** — is standard practice for any DP solution: the forward pass computes the *best score*, and a backward pass through a table of remembered choices reconstructs the *actual solution* that achieved it.

6.8 4.8 From Boundaries to a Usable Rating Function

Once `edges` (the list of boundary scores) exists, converting any raw FICO score into a rating is simple: find which bucket the score falls into, then convert the bucket's position into a rating number, flipping the order so that the *highest*-scoring bucket becomes rating 1 (best), per Charlie's requirement:

```
def fico_to_rating(score, edges=edges):
    num_buckets = len(edges) - 1
    for bucket_idx in range(num_buckets):
        lower, upper = edges[bucket_idx], edges[bucket_idx + 1]
        is_last = bucket_idx == num_buckets - 1
        if (lower <= score < upper) or (is_last and lower <= score <= upper):
            return num_buckets - bucket_idx
    raise ValueError(f"fico score {score} is outside 300-850")
```

`num_buckets - bucket_idx` is the flip: bucket 0 (the lowest-scoring range) is at the bottom of a 0-indexed list, but should map to the *highest* (worst) rating number, while the very last bucket (highest scores) should map to rating 1.

6.9 4.9 Checking the Result Makes Sense

The whole point of this exercise was to find boundaries where default risk differs meaningfully from one bucket to the next. Running the finished bucketing on the actual loan data produced results like:

[408-521]	n=316	default_rate=0.652	rating=10
[521-553]	n=515	default_rate=0.454	rating=9
[553-581]	n=964	default_rate=0.329	rating=8
[581-612]	n=1634	default_rate=0.240	rating=7
[612-650]	n=2538	default_rate=0.161	rating=6
[650-697]	n=2664	default_rate=0.096	rating=5
[697-733]	n=1124	default_rate=0.057	rating=4
[733-753]	n=311	default_rate=0.026	rating=3
[754-850]	n=242	default_rate=0.021	rating=1

This is exactly the pattern hoped for: default rate falls steadily and substantially as FICO score rises, from 65% in the worst bucket down to roughly 2% in the best. The buckets also aren't equal width — they're narrower at the risky, high-default end of the range (where getting the boundary right matters more) and wider at the safe end, exactly the benefit over naive equal-width bucketing described in section 4.2.

6.10 4.10 An Honest Caveat About Sparse Data

One tiny bucket appeared during testing — a range covering only 13 borrowers, with a default rate that didn't fit the otherwise clean downward trend. This isn't a bug in the code; it's a real, known limitation of optimizing log-likelihood directly: with very few borrowers in a thin slice of the score range, the observed default rate in that slice is statistically noisy (13 people is a small sample to draw firm conclusions from), and the optimizer can occasionally carve out an oddly-placed boundary chasing a pattern that's really just random noise rather than a genuine signal. A common refinement for a production version of this technique is to add a **minimum bucket size** constraint directly into the dynamic programming step, preventing the optimizer from ever proposing a bucket too small to be statistically trustworthy — a natural next step before this moves beyond prototype stage.

6.11 4.11 Aligning with the Course's Official Reference Answer

The official reference solution for this task uses the same core DP-and-log-likelihood idea, with a few implementation differences worth noting:

- It works across the **full official FICO range of 300–850**, rather than only the range actually present in the sample data (which happened to start at 408). This matters if the bucketing map ever needs to be applied to a different batch of loan data with a wider score spread.
- It builds the underlying counts using **running (cumulative) totals** indexed directly by score, computed with simple Python loops rather than pandas grouping — functionally equivalent, just a different style of implementation.
- Both arrive at the same underlying algorithm: dynamic programming over possible bucket cut-points, maximizing the same log-likelihood formula from section 4.3.

6.12 4.12 A Tiny, Fully-Worked Toy Example

The real problem involves roughly 550 possible scores and 10 buckets, which is too large to trace by hand. Here’s a drastically simplified toy version, with only 4 possible scores (call them positions 1, 2, 3, 4) and a target of 2 buckets, just to make the dynamic programming mechanics fully concrete.

Suppose the (made-up) data looks like this, where each position has some number of borrowers and some number of defaults among them:

Position	Borrowers (n)	Defaults (k)
1	10	8
2	10	5
3	10	2
4	10	1

To split this into 2 buckets, there are only 3 possible places to make the cut: after position 1, after position 2, or after position 3. A brute-force approach here is actually feasible by hand — calculate the log-likelihood (section 4.3) for each of the 3 possible splits, and keep whichever total is highest. Dynamic programming becomes valuable not because this specific tiny example needs it, but because the *same mechanical process* — try each possible last cut, combine it with the best score for everything before that cut, keep the winner — scales up cleanly to the real problem’s 550 positions and 10 buckets, where trying every combination directly would be computationally hopeless. The toy example above has only 3 candidate splits; the real FICO problem has an astronomically larger number of candidate combinations, which is precisely why the sub-problem-reuse trick from section 4.5 matters in practice, even though it’s overkill for a 4-position toy case.

Notice something else in the toy table: defaults drop steadily from position 1 (8 out of 10) to position 4 (1 out of 10) — a clean, monotonic pattern, similar in spirit to the real FICO results in section 4.9, where default rate fell steadily as score rose. A good 2-bucket split here would naturally separate the high-default positions from the low-default ones, exactly mirroring what the real algorithm does at much larger scale.

6.13 4.13 Frequently Asked Questions

Why choose 10 buckets specifically? The code is written so the number of buckets is just an input to `get_fico_buckets(num_buckets)` — 10 was chosen as a reasonable, round default for demonstration, but Charlie can call the same function with any number that matches whatever her specific model architecture expects as its number of input categories. More buckets means finer-grained risk separation but also smaller (and therefore noisier) sample sizes within each bucket, exactly the sparse-bucket trade-off discussed in section 4.10.

Is this the same technique used to build real-world credit rating scales (like AAA, AA, A, BBB...)? The conceptual goal is similar — group things with similar risk into a shared category — but real rating agencies combine statistical analysis like this with a great deal of qualitative judgment, industry expertise, and often deliberately smoothed or “through-the-cycle” adjustments so ratings don’t swing wildly with every small data update. This project’s approach is a simplified, purely data-driven version aimed specifically at feeding a machine learning model, not at producing a public-facing credit rating.

Could the MSE (mean squared error) approach mentioned in the brief have been used instead, and would it give a different answer? Yes — it’s the other option explicitly named in the task. The MSE approach would choose bucket boundaries to minimize the error from representing every score in a bucket by one single approximating value (e.g., the bucket’s average), treating this purely as a data-compression problem. It wouldn’t directly account for default rates at all — two buckets with wildly different default rates could get merged together as long as the raw scores were close together numerically. The log-likelihood approach used here was chosen specifically because it optimizes for the thing that actually matters for a credit model: separating groups with meaningfully different default behavior, not merely groups with numerically similar scores.

7 Part Five: How It All Fits Together

The four parts of this guide were built separately, but it's worth stepping back and walking through them as one continuous story — the way they'd actually unfold on a real desk.

7.1 5.1 The Gas Desk's Story, Start to Finish

Step one. A client tells Alex they believe the coming winter will be colder than usual, and they want to buy natural gas now, store it, and sell it in winter, betting the seasonal price rise this year will be larger than normal. Before the desk can quote this client anything, two questions need answers: *what will gas actually cost on the specific dates involved*, and *what is a fair, defensible price for the whole deal, once every fee is accounted for*.

Step two. The natural gas price model from Part One answers the first question. Given the historical monthly snapshots, it decomposes the pattern into a slow trend and a repeating seasonal cycle, and can now answer “what does gas cost” for any date the client proposes, whether that date already has real historical data behind it or falls somewhere in the year ahead that hasn't happened yet.

Step three. The storage contract pricing engine from Part Two answers the second question. Feed it the client's proposed injection dates, withdrawal dates, the prices from step two (or live quotes, if available), the facility's physical rate limits and capacity, and its fee structure, and it returns a single number: the fair value of the whole deal, accounting for every cash flow involved — not just the obvious buy-low-sell-high profit, but every fee that eats into it.

Step four. Alex now has a number to actually discuss with the client — not a gut feeling, but a figure built up transparently from real historical pricing behavior and a complete accounting of costs. If the client wants to try different dates, volumes, or contract lengths, the same two tools can be re-run instantly with the new numbers, letting Alex explore options interactively rather than requiring a fresh manual calculation every time.

7.2 5.2 The Credit Risk Story, Start to Finish

Step one. Charlie's team notices default rates on personal loans are running higher than expected. Before the bank can set aside the right amount of capital to absorb likely future losses, they need a systematic way to estimate, for any given borrower, how likely they are to default, and how much money is actually at stake.

Step two. The probability-of-default model from Part Three answers the first half of this. Trained on the bank's existing loan book — where the actual outcome (defaulted or not) for each historical borrower is already known — it learns the statistical relationship between a borrower's characteristics (income, existing debt, credit history, FICO score) and their likelihood of default, expressed as engineered debt and payment ratios rather than raw dollar figures, so the pattern generalizes across borrowers at very different income levels.

Step three. The expected loss formula turns that probability into an actual dollar figure the risk team can use for capital planning — multiplying the predicted probability by the loan amount at risk and by the fraction of that amount genuinely expected to be lost even after accounting for partial recovery.

Step four. Separately, Charlie mentions that a different model she's building for the mortgage book needs its FICO scores converted into clean categories rather than raw numbers. The bucketing tool

from Part Four solves exactly this, using the same underlying idea — statistical soundness driven by actual default patterns in the data, not an arbitrary or purely mechanical split — extended with a dynamic programming search to find the mathematically best possible set of category boundaries rather than a hand-picked guess.

7.3 5.3 The Common Thread

Despite covering two very different businesses (commodities trading and retail lending), all four pieces of work share the same underlying discipline:

1. **Start from real historical data**, not assumptions.
2. **Decompose or simplify the problem** into pieces that are individually well-understood and mathematically tractable — trend and seasonality; cash flows event by event; a probability multiplied by an exposure; a search over possible category boundaries.
3. **Build something that generalizes**, not something hard-coded to the one example given in the brief — a date-taking function rather than a single hard-coded number; a function that handles any number of injection/withdrawal dates rather than just one of each; a model that predicts for any borrower's characteristics rather than memorizing the training data; a bucketing function that works for any chosen number of buckets.
4. **Test against a known answer wherever one exists** — the \$580,000 worked example, a real row pulled from the loan data with a known outcome, a monotonically falling default rate across FICO buckets — before trusting the result.
5. **State the limitations honestly**. Every part of this guide includes an explicit caveat section, not as an afterthought, but because a prototype that quietly hides its own weak points is far more dangerous than one that states them plainly. That honesty is exactly what lets an engineering, risk, or model validation team pick up the work and know precisely where to focus their scrutiny before anything goes near production.

8 Glossary

AUC (Area Under the Curve) — A score between 0.5 and 1.0 measuring how well a classification model ranks positive cases (e.g., defaulters) above negative cases (e.g., non-defaulters). 1.0 is perfect; 0.5 is no better than random guessing. See section 3.8.

Backtracking — Walking backward through a table of remembered choices (built during a dynamic programming forward pass) to reconstruct the actual solution, not just its score. See section 4.7.

Bucket / Bucketing — Grouping a continuous range of values (like FICO scores 300–850) into a smaller number of discrete categories. See Part Four.

Classification — A prediction problem where the output is a category (e.g., default / no default) rather than a number that can take a wide range of values. See section 0.6.

Coefficient / Weight — A number learned by a model that describes how strongly a particular input feature influences the prediction. See section 3.5.

Commodity — A raw, physical good that is interchangeable regardless of who produced it (e.g., natural gas, crude oil, wheat). See section 0.1.

CSV (Comma-Separated Values) — A plain-text file format for storing tables of data, with commas separating columns and each line representing one row. See section 0.3.

DataFrame — pandas’ core object for representing a table of data inside a Python program, similar to a spreadsheet. See section 0.4.

Decision Tree — A predictive model that reaches an answer by asking a sequence of yes/no questions about the input data, like a flowchart. See section 3.7.

Default — When a borrower stops making the required payments on a debt. See section 0.2 and Part Three.

Dynamic Programming (DP) — A technique for solving a large problem by breaking it into smaller overlapping sub-problems, solving each one exactly once, and reusing those answers. See section 4.5.

Expected Loss (EL) — The amount of money a lender should, on average, expect to lose on a loan, calculated as $\text{Probability of Default} \times (1 - \text{Recovery Rate}) \times \text{Exposure}$. See section 3.2.

Exposure — The dollar amount actually at risk in a loan — typically the outstanding loan balance.

Extrapolation — Estimating a value beyond the range of data actually available (e.g., forecasting next year’s price using only past data). See section 0.7.

Feature Engineering — Deliberately transforming raw data columns into new, more informative inputs for a model (e.g., turning raw dollar amounts into debt-to-income ratios). See section 3.6.

FICO Score — A standardized U.S. credit score from 300 to 850 summarizing a borrower’s creditworthiness, used in the large majority of U.S. mortgage decisions. See section 4.1.

Function (in programming) — A named, reusable block of code that takes inputs and returns an output. See section 0.4.

Injection / Withdrawal (natural gas) — Injection is putting gas into underground storage (typically done in summer); withdrawal is taking it back out (typically done in winter). See section 0.2.

Interpolation — Estimating a value that falls between two data points that are already known. See section 0.7.

Linear Regression — A technique for fitting the best possible straight line through a set of data points, by minimizing the sum of squared vertical distances between the line and each point. See section 1.4.

Log-Likelihood — The logarithm of how statistically probable an observed outcome would be under a given model of the world; used here to judge how “internally consistent” a proposed FICO bucket is. See section 4.3.

Logistic Regression — A classification technique that fits a straight-line combination of inputs, then squeezes the result through the sigmoid function so the output is always a valid probability between 0 and 1. See section 3.5.

MMBtu — Million British Thermal Units; the standard unit natural gas is priced and traded in.

Model (predictive) — A mathematical rule for turning inputs into an output, where the rule’s exact parameters were learned from data. See section 0.5.

Overfitting — When a model learns to fit the specific quirks of its training data too closely, at the expense of generalizing well to new, unseen data. Mentioned in the context of decision trees, section 3.7.

pandas — A Python library specialized for working with tables of data (DataFrames). See section 0.4.

Parameter — A numeric value that defines a specific version of a general model shape (e.g., the slope and intercept of a specific line). See section 0.5.

Probability of Default (PD) — A number between 0 and 1 representing how likely a specific borrower is to default. See section 3.2.

Quantization — The general technique of mapping a continuous range of values into a smaller, discrete set of categories. See section 4.2.

Random Forest — A machine learning model built from many individual decision trees, trained on different random subsets of the data, whose predictions are averaged together to produce a more reliable overall prediction. See section 3.7.

Recovery Rate — The fraction of a defaulted loan’s value a lender typically manages to recover, even after default. See section 3.2.

Regression — A prediction problem where the output is a number that can take a wide range of values (e.g., a price). See section 0.6.

Residual — The difference between an actual observed value and what a model predicted for it. See section 1.5.

ROC Curve — A chart used to derive the AUC score, plotting a classification model’s true-positive rate against its false-positive rate across every possible decision threshold. See section 3.8.

scikit-learn — A Python library providing ready-made implementations of common machine learning algorithms, including logistic regression and random forests. See section 0.4.

Seasonality — A repeating pattern tied to a fixed calendar cycle (e.g., natural gas prices rising every winter and falling every summer). See section 1.2.

Sigmoid Function — A mathematical function that smoothly maps any real number into the range between 0 and 1, used to turn a raw linear score into a valid probability in logistic regression. See section 3.5.

Storage Contract — An agreement between a storage facility owner and a client to store a quantity of a commodity for a period of time, under agreed fees and limits. See section 0.2.

Trend — The slow, long-term drift in a series of values over time, separate from any short-term or seasonal fluctuation. See section 1.2.

9 Closing Notes

This guide walked through four connected but distinct pieces of work:

1. A **regression** model that decomposes historical natural gas prices into trend and seasonality, then uses that decomposition to interpolate historical prices and extrapolate one year forward.
2. A **cash-flow accounting engine** that turns a sequence of injection and withdrawal dates, prices, rates, and fees into a single dollar value for a storage contract.
3. A **classification** model (with two alternative techniques compared) that predicts a borrower's probability of default, converted into a dollar figure via the expected loss formula.
4. A **dynamic programming** optimization that converts a continuous FICO score into a small number of ordered rating categories, chosen to maximize how well those categories separate genuinely different levels of default risk.

Every one of these was explicitly described, in the original task briefs, as a **prototype** — a first working version meant to demonstrate a sound approach and get validated, stress-tested, and refined by engineering and risk teams before being trusted with real money or real regulatory capital decisions. That framing matters: the goal at this stage isn't a perfect, production-hardened system, but a clear, explainable, and *correct* first version that other people can understand, test, and build on.

The full, final code for all four pieces is collected in the appendix that follows, exactly as delivered.

10 Appendix: Full Code Listings

10.1 A.1 Natural Gas Price Estimation Model

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

# Load the monthly snapshot data
df = pd.read_csv("Nat_Gas.csv")
df["Dates"] = pd.to_datetime(df["Dates"], format="%m/%d/%y")
df = df.sort_values("Dates").reset_index(drop=True)
df["Prices"] = df["Prices"].astype(float)

first_date = df["Dates"].iloc[0]
last_date = df["Dates"].iloc[-1]

# Number of days since the first data point, used as our time axis
df["days"] = (df["Dates"] - first_date).dt.days

# Fit a simple straight line trend through the data (price vs time)
slope, intercept = np.polyfit(df["days"], df["Prices"], 1)

def trend_price(days):
    return slope * days + intercept

# Natural gas has a seasonal pattern - cheaper in summer when it's
# injected into storage, more expensive in winter when it's withdrawn
# for heating demand. To capture this, look at how far each month's
# actual price sits from the trend line, then average that by month.
df["trend"] = trend_price(df["days"])
df["residual"] = df["Prices"] - df["trend"]
df["month"] = df["Dates"].dt.month

monthly_avg = df.groupby("month")["residual"].mean()
monthly_avg = monthly_avg - monthly_avg.mean() # center around 0
seasonal_effect = monthly_avg.to_dict()

def get_seasonal(month):
    return seasonal_effect.get(month, 0)

def estimate_price(input_date):
    """
    Returns an estimated natural gas price for the given date.
    Works for any date between 2020-10-31 and one year past the
    last data point (2025-09-30).
    """
```

```

For dates we already have data around, it interpolates between
the nearest real data points. For future dates beyond the data,
it uses the trend line plus the seasonal adjustment for that month.
"""
d = pd.to_datetime(input_date)

cutoff = last_date + pd.DateOffset(years=1)
if d < first_date or d > cutoff:
    raise ValueError(f"Date must be between {first_date.date()} and {cutoff.date()}")

days = (d - first_date).days

if d <= last_date:
    return float(np.interp(days, df["days"], df["Prices"]))
else:
    return float(trend_price(days) + get_seasonal(d.month))

# quick check
for test_date in ["2021-06-15", "2023-12-25", "2024-09-30", "2025-01-15", "2025-09-30"]:
    print(test_date, "->", round(estimate_price(test_date), 2))

# build a daily series covering the data plus the extra forecast year, for plotting
all_dates = pd.date_range(first_date, last_date + pd.DateOffset(years=1), freq="D")
all_prices = [estimate_price(d) for d in all_dates]

plt.figure(figsize=(12, 6))
plt.scatter(df["Dates"], df["Prices"], color="navy", zorder=5, label="Actual monthly data")

hist_mask = all_dates <= last_date
plt.plot(all_dates[hist_mask], np.array(all_prices)[hist_mask], color="steelblue", label="Interpolated")
plt.plot(all_dates[~hist_mask], np.array(all_prices)[~hist_mask], color="firebrick", linestyle="dashed")

plt.axvline(last_date, color="gray", linestyle=":")
plt.title("Natural Gas Price - History and 1 Year Forecast")
plt.xlabel("Date")
plt.ylabel("Price")
plt.gca().xaxis.set_major_locator(mdates.YearLocator())
plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%Y"))
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("gas_price_forecast.png", dpi=150)

# separate plot just to see the seasonal shape
plt.figure(figsize=(8, 4.5))
months = list(range(1, 13))

```

```

labels = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]
values = [get_seasonal(m) for m in months]
plt.bar(labels, values, color=["firebrick" if v >= 0 else "steelblue" for v in values])
plt.axhline(0, color="black", linewidth=0.8)
plt.title("Seasonal Effect by Month")
plt.ylabel("$ vs trend")
plt.tight_layout()
plt.savefig("seasonal_profile.png", dpi=150)

```

ewpage

10.2 A.2 Storage Contract Pricing

```

from datetime import date
import math

# prices a gas storage contract - net value from buying, storing and selling gas
def price_contract(in_dates, in_prices, out_dates, out_prices, rate,
                  storage_cost_rate, total_vol, injection_withdrawal_cost_rate):

    volume = 0
    buy_cost = 0
    cash_in = 0

    # go through all dates in order so we process injections/withdrawals as they happen
    all_dates = sorted(set(in_dates + out_dates))

    for current_date in all_dates:
        if current_date in in_dates:
            # injecting gas - only if there's room left in storage
            if volume <= total_vol - rate:
                volume += rate
                buy_cost += rate * in_prices[in_dates.index(current_date)]
                injection_cost = rate * injection_withdrawal_cost_rate
                buy_cost += injection_cost
                print(f"Injected gas on {current_date} at a price of {in_prices[in_dates.index(current_date)]}")
            else:
                print(f"Injection is not possible on {current_date}, not enough space left in storage")

        elif current_date in out_dates:
            # withdrawing gas - only if there's enough currently stored
            if volume >= rate:
                volume -= rate
                cash_in += rate * out_prices[out_dates.index(current_date)]
                withdrawal_cost = rate * injection_withdrawal_cost_rate
                cash_in -= withdrawal_cost
                print(f"Extracted gas on {current_date} at a price of {out_prices[out_dates.index(current_date)]}")

```

```

        else:
            print(f"Extraction is not possible on {current_date}, not enough gas stored")

            # flat storage fee for the number of months the facility was in use
            store_cost = math.ceil((max(out_dates) - min(in_dates)).days / 30) * storage_cost_rate

            return cash_in - store_cost - buy_cost

# --- test it with a few sample dates, same shape as the course example ---
in_dates = [date(2022, 1, 1), date(2022, 2, 1), date(2022, 2, 21), date(2022, 4, 1)]
in_prices = [20, 21, 20.5, 22]
out_dates = [date(2022, 1, 27), date(2022, 2, 15), date(2022, 3, 20), date(2022, 6, 1)]
out_prices = [23, 19, 21, 25]

rate = 100_000 # cubic feet per day, injection or withdrawal
storage_cost_rate = 10_000 # flat $ per month to store gas
injection_withdrawal_cost_rate = 0.0005 # $ per cubic foot moved
max_storage_volume = 500_000 # cubic feet capacity

result = price_contract(in_dates, in_prices, out_dates, out_prices, rate,
                        storage_cost_rate, max_storage_volume, injection_withdrawal_cost_rate)

print()
print(f"The value of the contract is: ${result}")

# --- a second test, using my own dates/rates instead of the example ones ---
in_dates2 = [date(2024, 6, 30), date(2024, 7, 31)]
in_prices2 = [2.1, 2.3]
out_dates2 = [date(2024, 12, 31), date(2025, 1, 31)]
out_prices2 = [3.5, 3.8]

result2 = price_contract(in_dates2, in_prices2, out_dates2, out_prices2,
                        rate=500_000, storage_cost_rate=100_000,
                        total_vol=1_000_000, injection_withdrawal_cost_rate=0.01)

print()
print(f"Second test contract value: ${result2}")

```

ewpage

10.3 A.3 Loan Default Probability & Expected Loss Model

```

import pandas as pd
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn import metrics

```

```

# load the loan book
df = pd.read_csv("Task_3_and_4_Loan_Data.csv")

# rather than feeding in raw dollar amounts, use ratios - they tend to
# generalize better across borrowers with very different income levels
df["payment_to_income"] = df["loan_amt_outstanding"] / df["income"]
df["debt_to_income"] = df["total_debt_outstanding"] / df["income"]

features = ["credit_lines_outstanding", "debt_to_income",
            "payment_to_income", "years_employed", "fico_score"]

clf = LogisticRegression(random_state=0, solver="liblinear", tol=1e-5, max_iter=10000)
clf.fit(df[features], df["default"])

print("coefficients:", clf.coef_)
print("intercept:", clf.intercept_)

# quick check of how well it's doing on the training data itself
y_pred = clf.predict(df[features])
misclass_rate = (1.0 * (abs(df["default"] - y_pred)).sum()) / len(df)
fpr, tpr, thresholds = metrics.roc_curve(df["default"], y_pred)
auc = metrics.auc(fpr, tpr)

print("misclassification rate:", round(misclass_rate, 4))
print("AUC:", round(auc, 4))

RECOVERY_RATE = 0.10 # given in the brief

def predict_expected_loss(credit_lines_outstanding, loan_amt_outstanding,
                           total_debt_outstanding, income, years_employed, fico_score):
    """
    Takes the raw loan details for a borrower, builds the same ratio
    features the model was trained on, and returns the PD plus the
    expected loss on the loan.
    """
    payment_to_income = loan_amt_outstanding / income
    debt_to_income = total_debt_outstanding / income

    loan_features = pd.DataFrame([
        "credit_lines_outstanding": credit_lines_outstanding,
        "debt_to_income": debt_to_income,
        "payment_to_income": payment_to_income,
        "years_employed": years_employed,
        "fico_score": fico_score,
    ])

    pd_estimate = clf.predict_proba(loan_features)[:, 1][0]

```

```

expected_loss = pd_estimate * (1 - RECOVERY_RATE) * loan_amt_outstanding

return {
    "probability_of_default": round(float(pd_estimate), 4),
    "expected_loss": round(float(expected_loss), 2),
}

if __name__ == "__main__":
    # a borrower who looks fairly safe
    result1 = predict_expected_loss(
        credit_lines_outstanding=1,
        loan_amt_outstanding=3000,
        total_debt_outstanding=4000,
        income=70000,
        years_employed=6,
        fico_score=700,
    )
    print("\nBorrower 1 (looks low risk):", result1)

    # a borrower who looks riskier
    result2 = predict_expected_loss(
        credit_lines_outstanding=5,
        loan_amt_outstanding=8000,
        total_debt_outstanding=15000,
        income=28000,
        years_employed=1,
        fico_score=580,
    )
    print("Borrower 2 (looks higher risk):", result2)

    # sanity check against a real row from the data
    row = df.iloc[0]
    result3 = predict_expected_loss(
        credit_lines_outstanding=row["credit_lines_outstanding"],
        loan_amt_outstanding=row["loan_amt_outstanding"],
        total_debt_outstanding=row["total_debt_outstanding"],
        income=row["income"],
        years_employed=row["years_employed"],
        fico_score=row["fico_score"],
    )
    print(f"Borrower 3 (real row, actual default was {row['default']}):", result3)

```

ewpage

10.4 A.4 FICO Score Bucketing (Dynamic Programming)

```
import pandas as pd
import numpy as np

df = pd.read_csv("Task_3_and_4_Loan_Data.csv")

defaults = df["default"].to_list()
fico = df["fico_score"].to_list()
n = len(defaults)

# FICO scores officially run 300-850, so use that full range rather than
# just whatever happens to show up in this particular sample - that way
# the bucket map still makes sense if Charlie runs it on a different batch
# of loans later
MIN_SCORE = 300
MAX_SCORE = 850
span = MAX_SCORE - MIN_SCORE # 550

default_count = [0] * (span + 1)
total_count = [0] * (span + 1)

for i in range(n):
    score = int(fico[i])
    idx = score - MIN_SCORE
    default_count[idx] += defaults[i]
    total_count[idx] += 1

# turn these into running totals, so total_count[j] is "everyone with a
# score <= j+300", not just people at exactly that score
for i in range(1, span + 1):
    default_count[i] += default_count[i - 1]
    total_count[i] += total_count[i - 1]

def log_likelihood(n_in_bucket, k_defaults):
    if n_in_bucket == 0:
        return 0
    p = k_defaults / n_in_bucket
    if p == 0 or p == 1:
        return 0
    return k_defaults * np.log(p) + (n_in_bucket - k_defaults) * np.log(1 - p)

def get_fico_buckets(num_buckets):
    """
    dp[b][j] holds [best log-likelihood, index of the previous cut]
    for splitting scores 0..j (i.e. 300..j+300) into b buckets.
```

```

"""
dp = [[[-1e18, 0] for _ in range(span + 1)] for _ in range(num_buckets + 1)]

for b in range(num_buckets + 1):
    for j in range(span + 1):
        if b == 0:
            dp[b][j][0] = 0
        else:
            for cut in range(j):
                # skip cuts that don't actually separate any data
                if total_count[j] == total_count[cut]:
                    continue
                if b == 1:
                    dp[b][j][0] = log_likelihood(total_count[j], default_count[j])
                else:
                    candidate = dp[b - 1][cut][0] + log_likelihood(
                        total_count[j] - total_count[cut],
                        default_count[j] - default_count[cut],
                    )
                    if candidate > dp[b][j][0]:
                        dp[b][j][0] = candidate
                        dp[b][j][1] = cut

print("best log-likelihood:", round(dp[num_buckets][span][0], 4))

# walk backwards through the dp table to pull out where the cuts landed
boundaries = []
j = span
b = num_buckets
while b >= 0:
    boundaries.append(j + MIN_SCORE)
    j = dp[b][j][1]
    b -= 1

return sorted(set(boundaries))

edges = get_fico_buckets(10)
print("bucket edges:", edges)

def fico_to_rating(score, edges=edges):
    """Lower rating = better score, so the top bucket gets rating 1."""
    num_buckets = len(edges) - 1
    for bucket_idx in range(num_buckets):
        lower, upper = edges[bucket_idx], edges[bucket_idx + 1]
        is_last = bucket_idx == num_buckets - 1

```

```

        if (lower <= score < upper) or (is_last and lower <= score <= upper):
            return num_buckets - bucket_idx
        raise ValueError(f"fico score {score} is outside 300-850")

if __name__ == "__main__":
    print("\nbucket ranges and default rates:")
    for i in range(len(edges) - 1):
        lo, hi = edges[i], edges[i + 1]
        mask = (df["fico_score"] >= lo) & (df["fico_score"] <= hi)
        bucket_n = mask.sum()
        if bucket_n > 0:
            rate = df.loc[mask, "default"].mean()
            print(f" [{lo}-{hi}] n={bucket_n} default_rate={rate:.3f} rating={fico_to_rating}")

    print("\nsome example lookups:")
    for test_score in [320, 550, 600, 650, 700, 750, 820]:
        print(f" fico {test_score} -> rating {fico_to_rating(test_score)}")

```

11 Appendix B: Python & Library Quick Reference

This is a compact reference for every piece of Python and library syntax used across the four scripts in this guide, for looking up any single line without re-reading a whole chapter.

11.1 B.1 pandas

Code	What it does
<code>pd.read_csv("file.csv")</code>	Loads a CSV file into a DataFrame (a table).
<code>df["col"]</code>	Selects a single column from a DataFrame.
<code>df["col"] = ...</code>	Creates a new column, or overwrites an existing one.
<code>df.sort_values("col")</code>	Returns the DataFrame sorted by a given column.
<code>df.groupby("col")["other_col"].mean()</code>	Groups rows sharing the same value in <code>col</code> , and averages <code>other_col</code> within each group.
<code>pd.to_datetime(x)</code>	Converts a text date (or column of them) into a proper date/time object pandas understands.
<code>df.iloc[0]</code>	Selects the first row of a DataFrame by position.
<code>df.loc[mask, "col"]</code>	Selects values in <code>col</code> for only the rows where <code>mask</code> (a True/False condition) is True.

11.2 B.2 NumPy

Code	What it does
<code>np.polyfit(x, y, 1)</code>	Fits the best-fitting straight line (degree 1) through the points (<code>x</code> , <code>y</code>), returning slope and intercept.
<code>np.interp(x, known_x, known_y)</code>	Straight-line interpolation: estimates the y-value at <code>x</code> , given known (<code>x</code> , <code>y</code>) pairs.
<code>np.log(x)</code>	Natural logarithm.
<code>np.array(...)</code>	Converts a plain Python list into a NumPy array, enabling fast, vectorized math operations.
<code>np.full((a, b), value)</code>	Creates an <code>a</code> by <code>b</code> grid, filled entirely with <code>value</code> — used to initialize the dynamic programming tables in Part Four.

11.3 B.3 scikit-learn

Code	What it does
<code>LogisticRegression()</code>	Creates a logistic regression model (untrained).
<code>RandomForestClassifier()</code>	Creates a random forest model (untrained).

Code	What it does
<code>model.fit(X, y)</code>	Trains the model — <code>X</code> is the table of input features, <code>y</code> is the known correct answers.
<code>model.predict_proba(X)</code>	Returns predicted probabilities for each class (e.g., no-default vs. default) for each row in <code>X</code> .
<code>train_test_split(X, y, test_size=0.2)</code>	Randomly splits data into a training portion and a held-out test portion (see section 0.10).
<code>roc_auc_score(y_true, y_pred)</code>	Calculates the AUC score (section 3.8) comparing true outcomes to predicted probabilities.

11.4 B.4 Core Python

Code	What it does
<code>def function_name(arg1, arg2):</code> <code>return value</code>	Defines a new function taking two inputs. Sends a value back out of a function as its result.
<code>for x in some_list:</code>	Loops over every item in <code>some_list</code> , one at a time.
<code>if condition: ... else: ...</code>	Runs one block of code if <code>condition</code> is true, another if false.
<code>raise ValueError("message")</code>	Immediately stops the current function and reports an error with the given message.
<code>try: ... except ValueError as e: ...</code>	Attempts to run some code, and catches (handles gracefully) a <code>ValueError</code> if one occurs, instead of crashing.
<code>zip(list_a, list_b)</code>	Pairs up corresponding items from two lists, position by position — the first item of <code>list_a</code> with the first of <code>list_b</code> , and so on.
<code>sorted(x, key=lambda item: item[0])</code>	Sorts a collection, using whatever <code>item[0]</code> (the first element of each item) evaluates to as the sort key.
<code>f"text {variable}"</code>	An f-string — a text string with a variable's value automatically inserted wherever <code>{variable}</code> appears.
<code>dict.get(key, default)</code>	Looks up <code>key</code> in a dictionary, returning <code>default</code> instead of crashing if the key isn't found.